The Mathematics of Axial Tilt in Kerbal Space Program

Reference frames, the rotating-frame switch, and orbital elements

The reference-frame construction used by the Tilt'Em mod
for Kerbal Space Program 1.12

What this is. Kerbal Space Program has no axial tilt: every planet's north pole points along the same world axis, and the game's internal machinery quietly depends on that in more places than you would guess. This document develops, from first principles, the mathematics needed to introduce a real obliquity without breaking the machinery, and explains why several obvious approaches fail.

Required background. Matrix multiplication, orthogonal matrices, and the dot and cross products. Everything past that is introduced where it is first used: rotation groups, conjugation, Euler-angle parameterisations, floating-point conditioning, and what little KSP internals and celestial mechanics are needed. The prerequisites are light, but the argument is dense, and the whole is suited for upper-level undergraduate students.

On the listings. Code quoted from KSP is decompiled from the 1.12.5 assemblies, so its local names are not always the originals; where the decompiler emitted placeholders, they have been replaced with the names KSP itself uses for the same quantity elsewhere. Code quoted from the mod is verbatim.

Contents

1 Frames, and how KSP stores them

1.1 A frame is a matrix

Fix a reference basis for three-dimensional space. A **frame** is an ordered triple of mutually perpendicular unit vectors $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z})$, each written out in that reference basis, and ordered so that $\boldsymbol{x} \times \boldsymbol{y} = \boldsymbol{z}$.

Stack the three as the columns of a matrix:

$$\boldsymbol{M} = (\boldsymbol{x} \;\; \boldsymbol{y} \;\; \boldsymbol{z}) \in \mathbb{R}^{3\times 3}. \tag{1}$$

Because the columns are orthonormal, $\boldsymbol{M}^{\mathrm{T}}\boldsymbol{M} = \boldsymbol{I}$, so

$$\boldsymbol{M}^{-1} = \boldsymbol{M}^{\mathrm{T}}, \tag{2}$$

and because the triple is right-handed, $\det \boldsymbol{M} = +1$. Matrices with these two properties are exactly the **rotation matrices**, the group $\mathrm{SO}(3)$. So "frame", "orthonormal basis" and "rotation matrix" are three names for one object, and we use them interchangeably. Note the practical upshot of Equation 2: **inverting a frame costs nothing but a transpose.**

1.2 Two ways to read the same matrix

A rotation matrix can be read in two ways. They are easy to conflate, so it is worth separating them explicitly.

Active $\boldsymbol{M}$ takes a vector and **moves** it. The vector $\boldsymbol{v}$ becomes $\boldsymbol{M}\boldsymbol{v}$, somewhere else in the same space.

Passive $\boldsymbol{M}$ takes **coordinates** and **relabels** them. If a vector has coordinates $\boldsymbol{c}$ with respect to the frame's own axes, then its coordinates in the reference basis are $\boldsymbol{M}\boldsymbol{c}$. Nothing moved; we changed our description.

Both readings are the same arithmetic. Which one is meant is a matter of intent, and this document will always say which.

1.3 KSP's `CelestialFrame`

KSP stores a frame as its three columns, exactly as above:

```
public struct CelestialFrame
{
    public Vector3d X, Y, Z;

    public Vector3d LocalToWorld(Vector3d r)   // = M r
    {
        return r.x * X + r.y * Y + r.z * Z;
    }

    public Vector3d WorldToLocal(Vector3d r)   // = transpose(M) r
    {
        return new Vector3d(Dot(r, X), Dot(r, Y), Dot(r, Z));
    }
}
```

> **Warning.** The method names are the opposite of what a newcomer expects. Here **"World"** means **the basis the columns are written in**, and **"Local"** means **this frame's own axes**. So `WorldToLocal` is multiplication by $\boldsymbol{M}^{\mathrm{T}}$, and it converts **out of** the basis the frame lives in and **into** the frame. We will see in Section 3 that for the frame called `Planetarium.Zup`, "Local" is the coordinate system the game engine actually draws in.

1.4 The two elementary rotations

We only ever need rotations about the reference z- and x-axes:

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}. \tag{3}$$

Both are the ordinary right-handed active rotations: $R_z(\theta)$ sends $\boldsymbol{e}_1$ to $(\cos\theta, \sin\theta, 0)$, turning $+x$ towards $+y$ for positive θ. KSP's own generator is built from these two and nothing else, which Proposition 2.1 verifies.

Two facts used constantly:

$$R_z(\alpha)R_z(\beta) = R_z(\alpha+\beta), \quad R_z(\theta)^{\mathrm{T}} = R_z(-\theta). \tag{4}$$

That is, rotations about a **common** axis commute and simply add their angles. Rotations about **different** axes do neither. Propositions 5.4, 5.5 and 6.1 each turn on that asymmetry, so it is worth having in mind.

2 KSP's celestial frames share one generator

Every celestial frame that matters here — planet orientations, orbit planes, the planetarium itself — is built through a single function. Both public entry points below reduce to it, and every `PlanetaryFrame` call site in the stock assemblies goes through one of those two.

```
public static void SetFrame(double A, double B, double C,
                            ref CelestialFrame cf)
{
    cf.X = new Vector3d( cosA*cosC - sinA*cosB*sinC,
                         sinA*cosC + cosA*cosB*sinC,
                         sinB*sinC);
    cf.Y = new Vector3d(-cosA*sinC - sinA*cosB*cosC,
                        -sinA*sinC + cosA*cosB*cosC,
                         sinB*cosC);
    cf.Z = new Vector3d( sinA*sinB, -cosA*sinB, cosB);
}
```

That wall of trigonometry is not arbitrary.

> **Proposition 2.1.** `SetFrame`(A, B, C) stores the columns of
>
> $$\boldsymbol{S}(A,B,C) = R_z(A)R_x(B)R_z(C). \tag{5}$$

Proof. Multiply the right-hand side out. First,

$$R_x(B)R_z(C) = \begin{pmatrix} \cos C & -\sin C & 0 \\ \cos B\sin C & \cos B\cos C & -\sin B \\ \sin B\sin C & \sin B\cos C & \cos B \end{pmatrix}. \tag{6}$$

Left-multiplying by $R_z(A)$ mixes the first two rows in the usual way and leaves the third alone:

$$\boldsymbol{S} = \begin{pmatrix} \cos A\cos C - \sin A\cos B\sin C & -\cos A\sin C - \sin A\cos B\cos C & \sin A\sin B \\ \sin A\cos C + \cos A\cos B\sin C & -\sin A\sin C + \cos A\cos B\cos C & -\cos A\sin B \\ \sin B\sin C & \sin B\cos C & \cos B \end{pmatrix}. \tag{7}$$

Reading off the three **columns** — recall from Section 1.1 that a frame is stored as its columns — reproduces `cf.X`, `cf.Y` and `cf.Z` exactly. □

So KSP's generator is a **ZXZ Euler angle** parameterisation, one of several in use across celestial mechanics that differ in axis order and in sign. KSP commits to this one everywhere, for planetary orientations and orbital elements alike, which is what lets frames of the two kinds compose directly. Its third column will be needed repeatedly, so we record it:

$$\boldsymbol{S}(A,B,C)\boldsymbol{e}_3 = \begin{pmatrix} \sin A \sin B \\ -\cos A \sin B \\ \cos B \end{pmatrix}. \tag{8}$$

Note that the third column does not depend on C at all: C is a spin **about** that column.

2.1 The two public entry points

KSP exposes `SetFrame` through two thin wrappers that differ only by constant offsets:

$$\begin{aligned} \mathrm{OrbitalFrame}(\Omega, i, \omega) &= \boldsymbol{S}(\Omega, i, \omega), \\ \mathrm{PlanetaryFrame}(\alpha, \delta, r) &= \boldsymbol{S}(\alpha - 90°, \delta - 90°, r + 90°). \end{aligned} \tag{9}$$

Because they share a generator, **frames built by one compose directly with frames built by the other**, a fact we exploit in Section 8 to convert orbital elements between reference planes.

2.2 What the planetary offsets are for

Proposition 2.2. Write $T = \mathrm{PlanetaryFrame}(\alpha, \delta, 0)$. Then

$$T\boldsymbol{e}_3 = \begin{pmatrix} \cos\delta\cos\alpha \\ \cos\delta\sin\alpha \\ \sin\delta \end{pmatrix}, \tag{10}$$

which is the unit vector at right ascension α and declination δ.

Proof. Substitute $A = \alpha - 90°$ and $B = \delta - 90°$ into Equation 8, using $\sin(\alpha - 90°) = -\cos\alpha$, $\cos(\alpha - 90°) = \sin\alpha$, and likewise for δ:

$$T\boldsymbol{e}_3 = \begin{pmatrix} (-\cos\alpha)(-\cos\delta) \\ -(\sin\alpha)(-\cos\delta) \\ \sin\delta \end{pmatrix} = \begin{pmatrix} \cos\delta\cos\alpha \\ \cos\delta\sin\alpha \\ \sin\delta \end{pmatrix}. \tag{11}$$

□

This is why the offsets exist. `PlanetaryFrame` is a frame **named by where its third axis points**, in the astronomers' convention: right ascension and declination of the north pole. That is the same pole representation the IAU body-orientation models use, so published pole data converts into this form directly.

Using real IAU data. A full IAU model gives the pole as a function of time: the right ascension and the declination each carry a drift rate, and the prime meridian angle advances with the body's spin. T holds one fixed orientation, with no rate term anywhere in it, so the time dependence has to be collapsed before the numbers reach the game. That is done once, offline, when the planet configuration is written: evaluate the IAU expressions at a chosen instant, the **epoch**, and record the resulting α, δ and prime meridian as three constants. Every body in a system needs the same epoch, since a pole is only meaningful against the frame of the moment it was taken.

Proposition 2.3. $\mathrm{PlanetaryFrame}(\alpha, \delta, r) = T \cdot R_z(r)$, with T as above.

Proof. From Equation 9 and Equation 5,

$$\begin{aligned}\mathrm{PlanetaryFrame}(\alpha,\delta,r) &= R_z(\alpha-90^\circ)R_x(\delta-90^\circ)R_z(r+90^\circ)\\ &= \underbrace{R_z(\alpha-90^\circ)R_x(\delta-90^\circ)R_z(90^\circ)}_{T}\cdot R_z(r),\end{aligned} \tag{12}$$

where the last step splits $R_z(r+90^\circ) = R_z(90^\circ)R_z(r)$ using Equation 4. □

So a planetary frame factors cleanly into **a constant part carrying the pole** and **a variable part carrying the spin**. Section 5 works entirely in terms of that factorisation.

2.3 The degenerate case, and why stock KSP is stuck

Proposition 2.4. $\mathrm{PlanetaryFrame}(0, 90^\circ, r) = R_z(r)$, a pure spin about the reference z-axis.

Proof. Here $A = -90^\circ$, $B = 0$, $C = r + 90^\circ$. Since $R_x(0) = \boldsymbol{I}$,

$$\boldsymbol{S} = R_z(-90^\circ)R_z(r+90^\circ) = R_z(-90^\circ + r + 90^\circ) = R_z(r). \tag{13}$$

□

This has a consequence for the unmodified game:

Corollary 2.5. Stock KSP calls `PlanetaryFrame` with $(\alpha, \delta) = (0, 90^\circ)$ and **nothing else**, anywhere. Consequently every body's pole is $\boldsymbol{e}_3$, and obliquity is not merely absent: it is **structurally inexpressible** through the code path the game uses.

That is the constraint the whole mod is built around. We cannot bolt a tilt onto the output of `PlanetaryFrame` $(0, 90, r)$, because by Equation 10 the information has already been discarded. We must pass a real pole **into** the generator instead.

3 What the game does with these frames

Before adding tilt we need to know what the frames are actually consumed by. KSP does something here that has no counterpart in most engines.

3.1 The rotating frame

When the active vessel drops below a per-body altitude (`inverseRotThresholdAltitude`, 100 km at Kerbin), KSP flips that body into **inverse rotation**. Physically the motivation is straightforward. A vessel sitting on the launch pad of a body that rotates once per 21 549 s at a radius of 600 km is moving at about 175 m/s in world coordinates. The rotating frame exists to keep the local physics simulation from having to carry that motion, so that contacts between a landing leg and a 600 km collider mesh are resolved between two things at rest rather than two things travelling at 175 m/s.

So KSP stops the planet and turns the universe instead.

3.2 The two angles

Each body carries a spin phase that is pure physics, a function of time and nothing else:

$$\theta(t) = \left(\mathrm{initialRotation} + 360^\circ \cdot \frac{t}{P}\right) \bmod 360^\circ, \tag{14}$$

where t is universal time and P the body's rotation period, both in seconds, and `initialRotation` is the body's spin phase in degrees at $t = 0$. P and `initialRotation` are read from the body's configuration; t is the only variable. Alongside θ sit two bookkeeping quantities coupled by one invariant:

$$\theta = \theta_{\text{dir}} + \theta_{\text{inv}} \pmod{360^\circ}. \tag{15}$$

- θ_{dir} (`CelestialBody.directRotAngle`) is **per body**. It is the body's spin phase **as expressed in the game's world coordinates**: the heading of its prime meridian on screen.
- θ_{inv} (`Planetarium.InverseRotAngle`) is a **single global scalar**. It is how far the world coordinate system itself has been turned.

The frames built from them are

$$\boldsymbol{Z} = R_z(\theta_{\text{inv}}) \quad \text{(the planetarium frame)}, \qquad \boldsymbol{B} = R_z(\theta_{\text{dir}}) \quad \text{(the body frame)}. \tag{16}$$

The relevant source is short enough to quote in full:

```
rotationAngle = (initialRotation
                 + 360.0 * rotPeriodRecip * Planetarium.GetUniversalTime()) % 360.0;

if (!inverseRotation)
{
    directRotAngle = (rotationAngle - Planetarium.InverseRotAngle) % 360.0;
    Planetarium.CelestialFrame.PlanetaryFrame(
        0.0, 90.0, directRotAngle, ref BodyFrame);
    rotation = BodyFrame.Rotation.swizzle;
    bodyTransform.rotation = rotation;
}
else
{
    Planetarium.InverseRotAngle = (rotationAngle - directRotAngle) % 360.0;
    Planetarium.CelestialFrame.PlanetaryFrame(
        0.0, 90.0, Planetarium.InverseRotAngle, ref Planetarium.Zup);
    Planetarium.Rotation =
        QuaternionD.Inverse(Planetarium.Zup.Rotation).swizzle;
}
```

The switch changes only **which of the two is computed from the other**:

mode	body frame $\boldsymbol{B}$	planetarium frame $\boldsymbol{Z}$
inertial	$R_z(\theta_{\text{dir}})$, recomputed	$R_z(\theta_{\text{inv}})$, frozen
rotating	**never written at all**	$R_z(\theta_{\text{inv}})$, recomputed

Note the entry in bold. In the rotating branch the body frame is not written to the same value; it is not written **at all**. The planet is frozen by omission, wherever it last happened to be pointing.

3.3 Why the transition costs nothing

> **Proposition 3.1 (Free continuity).** At the instant `inverseRotation` flips, neither θ_{dir} nor θ_{inv} changes value. Hence both $\boldsymbol{B}$ and $\boldsymbol{Z}$ are literally unchanged across the switch, **and this holds for any function used to build them from those two angles.**

Proof. No arithmetic is applied to **either angle** at the transition. Something else does happen on that tick, and Section 3.5 returns to it: `setRotatingFrame` rewrites every loaded rigidbody's velocity. But it leaves θ_{dir} and θ_{inv} alone, and those are the only inputs the frames have. On the tick before, θ_{dir} was computed from Equation 15 and θ_{inv} left alone; on the tick after, the reverse. Both branches preserve Equation 15, and at the instant of the switch both variables hold the values they held a tick earlier. Since $\boldsymbol{B}$ and $\boldsymbol{Z}$ are functions of those variables alone, they too are unchanged, whatever those functions happen to be. □

The parenthetical is what the rest of the document relies on. Continuity at the threshold is a property of the invariant Equation 15 alone, not of the particular frames stock chooses to build, so $R_z(\cdot)$ may be replaced by any frame-generating function and continuity survives.

3.4 The scalar subtraction is a change of coordinates

> **Proposition 3.2.** In stock KSP, $\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}} \cdot R_z(\theta)$ holds identically, in both modes.

Proof. $\boldsymbol{Z}^{\mathrm{T}} R_z(\theta) = R_z(-\theta_{\mathrm{inv}}) R_z(\theta) = R_z(\theta - \theta_{\mathrm{inv}}) = R_z(\theta_{\mathrm{dir}}) = \boldsymbol{B}$, using Equation 4 and Equation 15. The invariant holds in both branches, so the identity does too. □

In the passive reading, **the $-\theta_{\mathrm{inv}}$ term is not a correction for something happening elsewhere. It is the change of coordinates from the fixed celestial frame into the turned world frame.** That is why every body performs the subtraction, not just the one holding the rotating frame. The other bodies are not being adjusted for a neighbour's behaviour; they are being re-expressed in the coordinate system everything is drawn in.

Proposition 3.2 is the identity we will generalise. Everything that goes wrong when tilt is introduced goes wrong because $R_z(-\theta_{\mathrm{inv}})$ is a scalar subtraction, and a scalar subtraction is only a change of coordinates when both rotations share an axis.

3.5 The rotation is real, not notational

The passive reading of Section 1.2 fits $\boldsymbol{B}$ exactly. $\boldsymbol{B}$ converts a body's orientation from one set of coordinates to another, and converting coordinates disturbs nothing.

It does not fit $\boldsymbol{Z}$. When $\boldsymbol{Z}$ changes, the game is not describing the same arrangement of the solar system in different coordinates; it is putting the solar system somewhere else. Four distinct mechanisms carry that motion into what the player sees.

1. **The planet's transform stops.** As noted, the rotating branch never writes `BodyFrame`, `rotation`, or `bodyTransform.rotation`. The body's transform is therefore not advanced by this path at all, so the surface stops turning in world coordinates, which is the whole objective.
2. **Every body's world position is pushed through $\boldsymbol{Z}$.** Each `Orbit` position accessor ends in `Planetarium.Zup.WorldToLocal(...)`, i.e. multiplication by $\boldsymbol{Z}^{\mathrm{T}}$, and the result is written straight into a transform:

```
// Orbit.cs -- position from the elements, in the celestial frame
Vector3d rCelestial = OrbitFrame.X * xPos + OrbitFrame.Y * yPos;
return Planetarium.Zup.WorldToLocal(rCelestial);   // -> game world frame

// OrbitDriver.cs -- and the setter writes bodyTransform.position
celestialBody.position = referenceBody.position + pos;
```

3. **The sky is spun.** `GalaxyCubeControl` and `SkySphereControl` both set `transform.rotation = Planetarium.Rotation * initRot`, where `Planetarium.Rotation` is $\boldsymbol{Z}^{-1}$.
4. **Rigidbody velocities are rewritten at the switch.** `OrbitPhysicsManager.setRotatingFrame` walks every unpacked part and applies $\boldsymbol{v} \leftarrow \boldsymbol{v} - \boldsymbol{\omega} \times \boldsymbol{r}$. A change of notation does not touch a physics engine.

Since every position picks up the same $\boldsymbol{Z}^{\mathrm{T}}$, the vector from a planet to its moon is rotated too. With the planet's orientation frozen, the moon now circles it once per planetary day, which is exactly what an observer on the ground sees either way. Same observable, opposite implementation.

> **Consequence.** A rotation of $\boldsymbol{Z}$ about the wrong axis is not a cosmetic error. It moves every body's transform, the terrain system, and the skybox.

4 Representing a tilt

4.1 Poles, not Euler angles

Reading a rotation matrix. Recall from Section 1.1 that a frame is stored as its columns. That gives a way to read any rotation matrix at a glance: **column k tells you where the k-th reference axis ends up.** Formally, multiplying by $\boldsymbol{e}_k$ selects the k-th column, so if $\boldsymbol{M} = (\boldsymbol{x}\ \ \boldsymbol{y}\ \ \boldsymbol{z})$ then $\boldsymbol{M}\boldsymbol{e}_1 = \boldsymbol{x}$, $\boldsymbol{M}\boldsymbol{e}_2 = \boldsymbol{y}$ and $\boldsymbol{M}\boldsymbol{e}_3 = \boldsymbol{z}$.

Applied to a planet, the third column is the axis it spins about. That is the one part of a body's orientation which does not change as it turns, so it is the natural thing to store, and Equation 10 says `PlanetaryFrame` already accepts it directly.

Right ascension and declination are the sky's longitude and latitude, in that order: declination is the latitude-like one. The declination δ measures the angle north of the celestial equator, running from $-90°$ to $+90°$; the right ascension α measures the angle around that equator from a fixed reference direction, running from $0°$ to $360°$. Any direction is fixed by the pair, and Equation 10 is exactly the conversion from the pair to Cartesian components. So we store obliquity the way astronomers do:

$$T := \mathrm{PlanetaryFrame}(\alpha, \delta, 0), \qquad \boldsymbol{p} := T\boldsymbol{e}_3, \qquad \varepsilon := 90° - \delta, \tag{17}$$

where α, δ are the right ascension and declination of the body's north pole. In words: T is the rotation that carries $\boldsymbol{e}_3$ onto the pole $\boldsymbol{p}$, and ε is the angle between the two.

For a concrete case, take $\alpha = 0°$ and $\delta = 66.56°$, so $\varepsilon = 23.44°$. Then Equation 10 gives

$$\boldsymbol{p} = \begin{pmatrix} \cos 66.56° \cdot \cos 0° \\ \cos 66.56° \cdot \sin 0° \\ \sin 66.56° \end{pmatrix} = \begin{pmatrix} 0.3977 \\ 0 \\ 0.9175 \end{pmatrix}, \tag{18}$$

a unit vector leaning $23.44°$ away from $\boldsymbol{e}_3$ towards $\boldsymbol{e}_1$. An untilted body has $\delta = 90°$, which gives $\boldsymbol{p} = \boldsymbol{e}_3$ and $T = \boldsymbol{I}$: the rotation that moves nothing.

> **A caution about the word "obliquity". Throughout this document "obliquity" means the angle between the body's spin pole and the celestial $+z$ axis**, which is $\varepsilon = 90° - \delta$. It is not necessarily the conventional spin-orbit obliquity. Astronomers usually reserve **obliquity** for the angle between a body's pole and its own **orbit** normal. The two agree only when the reference plane is that orbit plane. They nearly do in the stock Kerbol system, where every planet orbits close to $z = 0$; they need not in a system built from real ICRF data, where a planet can have $\delta = 90°$ and hence $\varepsilon = 0$ here, while carrying a substantial true obliquity in the inclination of its orbit instead.

Why a pole and not three Euler angles. An Euler triple also determines an orientation, but it determines it as a **composition**, leaving the pole implicit: to answer "where does this body's axis point" you must multiply the three rotations out first. Many different triples also give the same pole, so two configurations describing the same physical tilt need not look alike. Storing the pole makes the quantity every result below depends on immediate, and leaves exactly one degree of freedom unaccounted for.

That leftover is the third parameter, the **prime meridian offset** μ, carried alongside. A pole plus a spin angle is one parameterisation among many that give the same pole, and they disagree about where longitude zero sits; μ absorbs that difference so a tilt converted from an older format reproduces the old body frame exactly. We prove in Section 5.5 that μ cannot affect the planetarium frame.

4.2 The conjugation lemma

The following lemma is used repeatedly below. Informally it says: **to rotate about an awkward axis, carry the problem to where the axis is convenient, rotate there, and carry it back.** If $\boldsymbol{R}$ takes $\boldsymbol{u}$ to $\boldsymbol{Ru}$, then $\boldsymbol{R}^{\mathrm{T}}$ takes $\boldsymbol{Ru}$ back to $\boldsymbol{u}$. Reading the product right to left, the sandwich below undoes the carry, rotates by θ about $\boldsymbol{u}$ where that is easy, then redoes it.

> **Lemma 4.1 (Conjugation).** Let $\boldsymbol{R}$ be a rotation and let $\boldsymbol{R}_{\boldsymbol{u}}(\theta)$ denote rotation by θ about the unit axis $\boldsymbol{u}$. Then
>
> $$\boldsymbol{R}\,\boldsymbol{R}_{\boldsymbol{u}}(\theta)\,\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{R}_{\boldsymbol{Ru}}(\theta). \tag{19}$$

Sketch. Write $\boldsymbol{N} = \boldsymbol{R}\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{R}^{\mathrm{T}}$. It is a rotation, and it fixes $\boldsymbol{Ru}$, so that is its axis. On any unit $\boldsymbol{w}$ perpendicular to $\boldsymbol{u}$ it returns $\cos\theta\,(\boldsymbol{Rw}) + \sin\theta\,(\boldsymbol{Ru}\times\boldsymbol{Rw})$, which is the planar rotation by θ about $\boldsymbol{Ru}$ — sign included, so the sense is settled as well as the magnitude. The full argument is in Appendix B.

> **Corollary 4.2.** $T\,R_z(\theta)\,T^{\mathrm{T}}$ is the rotation by θ **about the body's own pole** $\boldsymbol{p}$.

Proof. $R_z(\theta) = \boldsymbol{R}_{\boldsymbol{e}_3}(\theta)$, and $T\boldsymbol{e}_3 = \boldsymbol{p}$ by Equation 17. Apply Lemma 4.1. □

This is the tool that lets us turn the sky about a tilted axis using only the R_z-shaped generator KSP gives us.

5 The construction

Two conventions before anything else. Throughout this section θ is the body's **absolute** spin phase from Equation 14, never θ_{dir}, and e denotes **elapsed rotation**: how far the body has turned since it entered the rotating frame.

Why there is an anchor. Stock needs no such thing, because its planetarium frame is driven by θ_{inv}, a global accumulator that is never reset. It holds the total turn, so $R_z(\theta_{\mathrm{inv}})$ is already an absolute answer. Three things stop us reusing it. It is a scalar about $\boldsymbol{e}_3$, and Section 6 shows that a tilted body needs the sky turned about $\boldsymbol{p}$ instead. It is global while the pole is per body, so its value means something different the moment the dominant body changes. And it is maintained by code the mod does not control: Kopernicus sets `inverseRotation` directly, skipping the update that keeps Equation 15 true.

So the planetarium frame here is driven by **elapsed** rotation e, computed from the body's own θ and nothing else. That independence costs something. Elapsed rotation is a relative quantity: it says how far the body has turned, not where it started, and a frame needs both. The anchor supplies the missing half. In the product $T\,R_z(e)\,T^{\mathrm{T}}\,\boldsymbol{A}$ the conjugated spin is the **how far** and $\boldsymbol{A}$ is the **from where**, which is to say $\boldsymbol{A}$ is the constant of integration. Stock got its equivalent for free by never resetting the accumulator.

With that settled, the construction is five objects, and the rest of this section proves they do what is required:

$$\begin{aligned}
T &= \mathrm{PlanetaryFrame}(\alpha, \delta, 0) && \text{(tilt frame, constant per body)} \\
\boldsymbol{L}(\theta) &\coloneqq T\,R_z(\theta + \mu) && \text{(local body frame)} \\
\boldsymbol{B} &\coloneqq \boldsymbol{Z}^{\mathrm{T}}\,\boldsymbol{L}(\theta) && \text{(body frame, world coordinates)} \\
\boldsymbol{Z}(e) &\coloneqq T\,R_z(e)\,T^{\mathrm{T}}\,\boldsymbol{A} && \text{(planetarium frame)} \\
\boldsymbol{A} &\coloneqq \boldsymbol{L}(\theta_0)\,\boldsymbol{C}^{\mathrm{T}} && \text{(anchor, latched on entry)}
\end{aligned} \tag{20}$$

The first line is Equation 17 repeated, not a new definition: T is fixed once per body by its pole, with $T\boldsymbol{e}_3 = \boldsymbol{p}$, and $T = \boldsymbol{I}$ when the body is untilted. It carries no time dependence, so every occurrence of T below is a constant matrix, and μ beside it is the prime meridian offset from the same place. Of the rest, θ_0 is the spin phase at the instant the body entered the rotating frame, and $\boldsymbol{C}$ is the body frame it had at that instant.

Compare these with what stock builds. $\boldsymbol{L}$ is Equation 10′s factorisation with a real pole instead of $\boldsymbol{e}_3$; $\boldsymbol{B}$ is Proposition 3.2 with the scalar subtraction written out as an honest matrix; $\boldsymbol{Z}$ is a spin, but conjugated by T so that Corollary 4.2 applies. Line by line, stock's split of one **angle** into two has become a split of one **rotation** into two:

	stock, rotating branch	**this construction**
the sky	$\boldsymbol{Z} = R_z(\theta_{\text{inv}})$, advancing	$\boldsymbol{Z}(e) = T R_z(e) T^{\mathrm{T}} \boldsymbol{A}$, advancing
the body	$\boldsymbol{B} = R_z(\theta_{\text{dir}})$, frozen	$\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}} \boldsymbol{L}(\theta)$, constant
the tie	$\theta = \theta_{\text{dir}} + \theta_{\text{inv}}$	$\boldsymbol{Z}(e)^{\mathrm{T}} \boldsymbol{L}(\theta_0 + e)$ free of e
the origin	θ_{inv} never resets	e resets on entry; $\boldsymbol{A}$ restores it

The right-hand column is the same idea carried out with matrices instead of a scalar, which is what it takes once the two rotations no longer share an axis. Where stock holds the body still by freezing a variable, Equation 20 holds it still by making $\boldsymbol{B}$ come out constant, and that is the content of Theorem 5.1.

5.1 The body frame is genuinely frozen

The rotating frame exists so that the ground does not move. Here is the proof that it does not.

Theorem 5.1 (Frozen body). With the definitions Equation 20, $\boldsymbol{B}$ is independent of e. Explicitly,

$$\boldsymbol{B} = \boldsymbol{A}^{\mathrm{T}} \boldsymbol{L}(\theta_0) \quad \text{for all } e. \tag{21}$$

Proof. During the rotating phase the spin phase is $\theta = \theta_0 + e$, so by Equation 4

$$\boldsymbol{L}(\theta_0 + e) = T\, R_z(\theta_0 + \mu + e) = T\, R_z(\theta_0 + \mu)\, R_z(e) = \boldsymbol{L}(\theta_0)\, R_z(e). \tag{22}$$

Also $\boldsymbol{Z}(e)^{\mathrm{T}} = \boldsymbol{A}^{\mathrm{T}} T\, R_z(-e)\, T^{\mathrm{T}}$, since $T^{\mathrm{T}} = T^{-1}$ and the transpose reverses the product. Therefore

$$\begin{aligned} \boldsymbol{B}(e) &= \boldsymbol{Z}(e)^{\mathrm{T}}\, \boldsymbol{L}(\theta_0 + e) \\ &= \boldsymbol{A}^{\mathrm{T}} T\, R_z(-e)\, T^{\mathrm{T}} \cdot \boldsymbol{L}(\theta_0)\, R_z(e) \\ &= \boldsymbol{A}^{\mathrm{T}} T\, R_z(-e)\, \underbrace{T^{\mathrm{T}} T}_{\boldsymbol{I}}\, R_z(\theta_0 + \mu)\, R_z(e) \quad [\text{using } \boldsymbol{L}(\theta_0) = T R_z(\theta_0 + \mu)] \\ &= \boldsymbol{A}^{\mathrm{T}} T\, R_z(-e)\, R_z(\theta_0 + \mu)\, R_z(e) \\ &= \boldsymbol{A}^{\mathrm{T}} T\, R_z(\theta_0 + \mu) = \boldsymbol{A}^{\mathrm{T}} \boldsymbol{L}(\theta_0), \end{aligned} \tag{23}$$

where the second-to-last step used the fact that rotations about a common axis commute and add, so $-e$ and $+e$ cancel. No e remains. □

Look at where the cancellation comes from. The body's own spin $R_z(e)$ appears on the **right** of $\boldsymbol{L}$; the conjugated sky rotation contributes $R_z(-e)$, and because it sits **inside** the conjugation — sandwiched between T^{T} and T — the T's annihilate and the two R_z's meet. They meet only because they are rotations about a common axis, which is what Equation 4 buys and what conjugating by T was arranged to preserve.

5.2 Choosing the anchor

Theorem 5.1 says $\boldsymbol{B}$ is **some** constant. Which constant is fixed entirely by $\boldsymbol{A}$, and we get to choose it.

Corollary 5.2. Setting $\boldsymbol{A} = \boldsymbol{L}(\theta_0)\boldsymbol{C}^{\mathrm{T}}$ gives $\boldsymbol{B} = \boldsymbol{C}$ throughout the rotating phase.

Proof. By Theorem 5.1, $\boldsymbol{B} = \boldsymbol{A}^{\mathrm{T}} \boldsymbol{L}(\theta_0)$. Substituting, $\boldsymbol{A}^{\mathrm{T}} = \boldsymbol{C}\, \boldsymbol{L}(\theta_0)^{\mathrm{T}}$, so $\boldsymbol{B} = \boldsymbol{C}\, \boldsymbol{L}(\theta_0)^{\mathrm{T}} \boldsymbol{L}(\theta_0) = \boldsymbol{C}$. □

So the anchor is not a free parameter to be tuned: it is **derived** from the orientation the body already has, and within Equation 20 it is the unique choice that leaves the body where it stands. In code:

```
public static Planetarium.CelestialFrame AnchorFor(BodyTilt tilt, double rot,
    Planetarium.CelestialFrame current, Planetarium.CelestialFrame zup)
{
    if (!IsUsableRotation(current)) return OrIdentity(zup);

    var local = default(Planetarium.CelestialFrame);
    LocalBodyFrame(tilt, rot, ref local);

    return Multiply(local, Transpose(current));   //  L(rot) * transpose(C)
}
```

5.3 Continuity at the threshold

Proposition 5.3. Suppose the body was inertial immediately before the switch, so that its frame was $\boldsymbol{C} = \boldsymbol{Z}_{\text{prev}}^{\mathrm{T}}\, \boldsymbol{L}(\theta_0)$ for the planetarium frame $\boldsymbol{Z}_{\text{prev}}$ in force at that moment. Then $\boldsymbol{A} = \boldsymbol{Z}_{\text{prev}}$, and $\boldsymbol{Z}(0) = \boldsymbol{Z}_{\text{prev}}$.

Proof.

$$\boldsymbol{A} = \boldsymbol{L}(\theta_0)\boldsymbol{C}^{\mathrm{T}} = \boldsymbol{L}(\theta_0)\big(\boldsymbol{Z}_{\text{prev}}^{\mathrm{T}}\boldsymbol{L}(\theta_0)\big)^{\mathrm{T}} = \boldsymbol{L}(\theta_0)\boldsymbol{L}(\theta_0)^{\mathrm{T}}\boldsymbol{Z}_{\text{prev}} = \boldsymbol{Z}_{\text{prev}}. \tag{24}$$

And $\boldsymbol{Z}(0) = T\, R_z(0)\, T^{\mathrm{T}}\boldsymbol{A} = TT^{\mathrm{T}}\boldsymbol{A} = \boldsymbol{A}$. □

Both frames are therefore continuous across the threshold, which is Proposition 3.1 realised for our particular choice of generator. Note that the anchor construction is **also** correct when $\boldsymbol{C}$ and $\boldsymbol{Z}_{\text{prev}}$ are inconsistent, since Corollary 5.2 does not assume they agree. That robustness matters: KSP has at least one code path (`PSystemSetup.SetSpaceCentre`) that flips a body into the rotating frame and then captures a floating-origin offset from its transform, at a moment when the body's frame is stale by a whole game session's worth of rotation. Deriving the anchor from $\boldsymbol{C}$ rather than capturing $\boldsymbol{Z}$ makes that failure impossible by construction.

5.4 Why the anchor multiplies on the right

Proposition 5.4. Defining instead $\tilde{\boldsymbol{Z}}(e) \coloneqq \boldsymbol{A}\, T\, R_z(e)\, T^{\mathrm{T}}$ — the anchor on the left — does **not** freeze the body frame.

Proof. $\tilde{\boldsymbol{Z}}(e)^{\mathrm{T}} = T\, R_z(-e)\, T^{\mathrm{T}}\, \boldsymbol{A}^{\mathrm{T}}$, so

$$\tilde{\boldsymbol{B}}(e) = T\, R_z(-e)\, T^{\mathrm{T}}\, \boldsymbol{A}^{\mathrm{T}}\, \boldsymbol{L}(\theta_0)\, R_z(e). \tag{25}$$

The factor $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{L}(\theta_0)$ sits between T^{T} and $R_z(e)$, so T^{T} has nothing to cancel against and the two $R_z(\pm e)$ never meet. The expression therefore depends on e unless $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{L}(\theta_0)$ commutes with every $R_z(e)$, which happens exactly when it is itself a spin about $\boldsymbol{e}_3$.

That factor has a name. Substituting the anchor of Corollary 5.2,

$$\boldsymbol{A}^{\mathrm{T}}\, \boldsymbol{L}(\theta_0) = \boldsymbol{C}\, \boldsymbol{L}(\theta_0)^{\mathrm{T}}\, \boldsymbol{L}(\theta_0) = \boldsymbol{C}, \tag{26}$$

so the left-anchored form freezes the body exactly when the frame it was latched against is a spin about $\boldsymbol{e}_3$. □

An untilted body satisfies that condition always, since then $\boldsymbol{L}$ and $\boldsymbol{C}$ are both plain R_z factors. So can a tilted one, if $\boldsymbol{C}$ happens to come out as a spin about $\boldsymbol{e}_3$. The left anchor is therefore not always wrong; it is

conditionally right, on a condition that turns on where the body happened to be pointing when the frame was taken. Equation 20 holds whatever $\boldsymbol{C}$ is.

The placement is therefore forced by the requirement, not chosen for convenience.

5.5 The prime meridian is invisible to the sky

Proposition 5.5. Folding μ into T — replacing T by $T\,R_z(\mu)$ in the definition of $\boldsymbol{Z}$ — changes nothing.

Proof.

$$(T R_z(\mu))\, R_z(e)\, (T R_z(\mu))^{\mathrm{T}} = T\, R_z(\mu)\, R_z(e)\, R_z(-\mu)\, T^{\mathrm{T}} = T\, R_z(e)\, T^{\mathrm{T}}, \tag{27}$$

since rotations about $\boldsymbol{e}_3$ commute by Equation 4. □

The sharper way to see it is through the axis rather than the algebra. By Lemma 4.1 each side is a rotation by e about the image of $\boldsymbol{e}_3$ under the conjugating frame, and

$$(T\, R_z(\mu))\boldsymbol{e}_3 = T\boldsymbol{e}_3 = \boldsymbol{p}, \tag{28}$$

so both conjugations turn about the very same axis. A spin about the pole cannot move the pole, so it cannot change what the conjugation produces.

Geometrically: μ is a spin about the pole, and a spin about an axis cannot move that axis. Conjugation only ever cares where the axis went. The mod therefore stores μ separately and applies it only inside $\boldsymbol{L}$, where it belongs.

5.6 Reduction to stock

Proposition 5.6. If $T = \boldsymbol{I}$ and $\mu = 0$, the definitions Equation 20 reduce to stock KSP's own **expressions**: $\boldsymbol{B} = R_z(\theta - \theta_{\mathrm{inv}})$ and $\boldsymbol{Z} = R_z(\theta_{\mathrm{inv}})$.

Proof. With $T = \boldsymbol{I}$, $\boldsymbol{L}(\theta) = R_z(\theta)$ and $\boldsymbol{Z}(e) = R_z(e)\boldsymbol{A}$. If the anchor was latched when the global angle was θ^0_{inv}, then $\boldsymbol{A} = R_z(\theta^0_{\mathrm{inv}})$ and $\boldsymbol{Z}(e) = R_z(\theta^0_{\mathrm{inv}} + e) = R_z(\theta_{\mathrm{inv}})$, because in stock θ_{inv} advances by exactly the elapsed rotation while θ_{dir} is frozen. Then $\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{L}(\theta) = R_z(-\theta_{\mathrm{inv}})R_z(\theta) = R_z(\theta - \theta_{\mathrm{inv}})$. □

One difference survives even at $T = \boldsymbol{I}$. While a body holds the rotating frame stock does not write `BodyFrame` at all, where Equation 20 recomputes it every tick. Theorem 5.1 makes the recomputed value constant, so the two agree on what the frame **is** while disagreeing on how often it is written. That is not a distinction without a difference: anything that reads the body's orientation between the two writes, a floating-origin offset taken from the body's transform above all, sees one version where stock would have shown it the other.

Subject to that, this is why the patch is safe to leave enabled on an untilted install: it is not merely **close** to stock there, it is the same expression.

6 Why the obvious approach fails, and why this one does not

The natural first attempt is to keep stock's arithmetic and multiply a tilt onto the result:

$$\boldsymbol{B}_{\mathrm{naive}} = T\, R_z(\theta - \theta_{\mathrm{inv}}) \quad \text{versus} \quad \boldsymbol{B} = R_z(-\theta_{\mathrm{inv}})\, T\, R_z(\theta). \tag{29}$$

These differ, and the difference is instructive.

Proposition 6.1. $\boldsymbol{B}_{\mathrm{naive}} = \boldsymbol{B}$ for all θ_{inv} if and only if T commutes with every R_z, i.e. if and only if the pole lies on $\pm\boldsymbol{e}_3$.

Proof. $\boldsymbol{B}_{\text{naive}} = T R_z(-\theta_{\text{inv}}) R_z(\theta)$ and $\boldsymbol{B} = R_z(-\theta_{\text{inv}}) T R_z(\theta)$. Cancelling the common right factor $R_z(\theta)$, equality for all θ_{inv} is exactly the statement that $T R_z(\varphi) = R_z(\varphi) T$ for all φ, i.e. that T commutes with the one-parameter group of rotations about $\boldsymbol{e}_3$. A rotation commutes with all rotations about an axis precisely when it preserves that axis. □

The failure has a clean geometric reading. Look at where each version puts the pole:

$$\boldsymbol{B}_{\text{naive}} \boldsymbol{e}_3 = T \boldsymbol{e}_3 = \boldsymbol{p}, \qquad \boldsymbol{B} \boldsymbol{e}_3 = R_z(-\theta_{\text{inv}}) \boldsymbol{p}. \tag{30}$$

The correct version turns the pole by $-\theta_{\text{inv}}$, which is right: the entire world frame was turned by $+\theta_{\text{inv}}$, so in world coordinates the pole must appear turned back by the same amount. The naive version leaves the pole exactly where it was, because $R_z(\theta - \theta_{\text{inv}})$ multiplies on the **right**, and by Proposition 2.3 a right factor is a spin about the pole, which can never move it.

Proposition 6.2 (Size of the error). The angle between the two poles is

$$\Delta = 2 \arcsin(\sin \varepsilon \cdot |\sin(\theta_{\text{inv}}/2)|), \tag{31}$$

where ε is the obliquity.

Proof. Both vectors lie on the cone of half-angle ε about $\boldsymbol{e}_3$, so both lie on a circle of radius $\sin \varepsilon$. Rotating by θ_{inv} moves a point of that circle by a chord of length $2 \sin \varepsilon |\sin(\theta_{\text{inv}}/2)|$. For unit vectors, chord c corresponds to angle $2 \arcsin(c/2)$. □

Two things follow. First the magnitude: Kerbin's legacy tilt has $\varepsilon = 20.591°$, so at $\theta_{\text{inv}} = 180°$ the pole is misplaced by $\Delta = 41.2°$. Second, and worse, θ_{inv} **changes continuously** while a body holds the rotating frame. The body's obliquity direction is therefore not merely wrong but drifting: a planet whose axis wanders as a vessel flies around it. In the game this showed up as an inverted season: a body reading its north pole as leaning towards the Sun in its southern midsummer.

Now put the same question to Equation 20. Where does it send the pole?

Corollary 6.3. Under Equation 20 the body's pole in world coordinates is $\boldsymbol{A}^{\mathrm{T}} \boldsymbol{p}$, which does not depend on the elapsed rotation e.

Proof. The pole in world coordinates is $\boldsymbol{B} \boldsymbol{e}_3$, and $\boldsymbol{L}(\theta) \boldsymbol{e}_3 = T\, R_z(\theta + \mu) \boldsymbol{e}_3 = T \boldsymbol{e}_3 = \boldsymbol{p}$, since R_z fixes $\boldsymbol{e}_3$. So $\boldsymbol{B} \boldsymbol{e}_3 = \boldsymbol{Z}^{\mathrm{T}} \boldsymbol{p}$, and

$$\boldsymbol{Z}(e)^{\mathrm{T}} \boldsymbol{p} = \boldsymbol{A}^{\mathrm{T}}\, T\, R_z(-e)\, T^{\mathrm{T}} \boldsymbol{p} = \boldsymbol{A}^{\mathrm{T}}\, T\, R_z(-e) \boldsymbol{e}_3 = \boldsymbol{A}^{\mathrm{T}}\, T \boldsymbol{e}_3 = \boldsymbol{A}^{\mathrm{T}} \boldsymbol{p}, \tag{32}$$

using $T^{\mathrm{T}} \boldsymbol{p} = \boldsymbol{e}_3$ and then that $R_z(-e)$ fixes $\boldsymbol{e}_3$. It also follows at once from Theorem 5.1, a constant $\boldsymbol{B}$ having a constant third column; the value of doing it directly is the comparison below. □

Three constructions, one question, three answers:

construction	**pole in world coordinates**
naive, $\boldsymbol{B}_{\text{naive}} = T R_z(\theta - \theta_{\text{inv}})$	$\boldsymbol{p}$, never moves, though the world frame turns
stock's $\boldsymbol{Z}$ with a real pole	$R_z(-\theta_{\text{inv}}) \boldsymbol{p}$, dragged about the wrong axis, drifting
Equation 20	$\boldsymbol{A}^{\mathrm{T}} \boldsymbol{p}$, fixed for as long as the frame is held

The reason the third row works is physical rather than algebraic. While a body is inertial its own spin supplies the sky's apparent motion, and a spin about an axis cannot move that axis. When the rotating frame takes over, the sky has to reproduce that motion exactly, so the sky's rotation must leave the axis alone too.

Conjugating by T is what makes R_z turn about $\boldsymbol{p}$ instead of $\boldsymbol{e}_3$, and by Lemma 4.1 a rotation about $\boldsymbol{p}$ fixes $\boldsymbol{p}$. Stock satisfies the same requirement, but only because every pole is $\boldsymbol{e}_3$ there, so the two axes coincide and the question never arises.

The crossing, end to end. A vessel descends past the threshold on a tilted planet. `inverseRotation` flips, and nothing else changes on that tick (Proposition 3.1). The anchor is latched as $\boldsymbol{A} = \boldsymbol{L}(\theta_0)\boldsymbol{C}^{\mathrm{T}}$, which by Corollary 5.2 leaves the body exactly where it stands and by Proposition 5.3 leaves the sky exactly where it was. While the vessel remains below, the body's frame does not move at all (Theorem 5.1), so the physics engine sees a stationary planet; the sky turns instead, about the body's own pole, so the obliquity cannot drift (Corollary 6.3) and the seasons and sub-solar point stay right however long the vessel stays down. On the way back out the anchor is released and the body resumes from its own θ. If the planet is untilted, every one of these expressions is stock's own (Proposition 5.6).

7 Worked examples

Everything so far has been symbolic. This section runs the construction on Kerbin's own tilt and gives the numbers, so each result above can be checked against a value rather than taken on the algebra alone.

All of them are produced by the shipped `TiltEmFrames`, not computed by hand, and can be regenerated with

```
dotnet run --project Docs/verification/TiltEmVerify.csproj -- --examples
```

Throughout, the body is Kerbin with the mod's default tilt, its spin phase at the moment of interest is $\theta_0 = 137.4°$, and the sky has already been turned by $\theta_{\mathrm{inv}} = 100°$ when the vessel drops below the threshold.

7.1 The tilt frame

Kerbin's tilt ships in the older format, as Unity Euler angles $(20, 0, 5)$. Converted to a pole by `FromLegacyEuler`:

quantity	value
α, right ascension of the pole	104.348568°
δ, declination of the pole	69.409328°
μ, prime meridian offset	−103.466390°
$\varepsilon = 90° - \delta$, obliquity	20.590672°

and $T = \mathrm{PlanetaryFrame}(\alpha, \delta, 0)$ comes out as

$$T = \begin{pmatrix} -0.231989 & -0.968806 & -0.087156 \\ 0.906916 & -0.247820 & 0.340719 \\ -0.351689 & 0.000000 & 0.936117 \end{pmatrix}. \tag{33}$$

The third column is the pole $\boldsymbol{p}$, and Equation 10 reproduces it from α and δ directly. The obliquity is the angle between that column and $\boldsymbol{e}_3$: $\arccos(0.936117) = 20.590672°$, agreeing with $90° - \delta$ as it must.

7.2 Entering the rotating frame

Before the crossing the body is inertial, so $\boldsymbol{Z} = R_z(100°)$ and the body frame is $\boldsymbol{C} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{L}(\theta_0)$. In columns, the pole sits at

$$\boldsymbol{L}(\theta_0)\boldsymbol{e}_3 = \begin{pmatrix} -0.087156 \\ 0.340719 \\ 0.936117 \end{pmatrix}, \qquad \boldsymbol{C}\boldsymbol{e}_3 = \begin{pmatrix} 0.350677 \\ 0.026666 \\ 0.936117 \end{pmatrix}. \tag{34}$$

Both have the same third component, since $R_z(-100°)$ turns the pole about $\boldsymbol{e}_3$ and cannot change its height. Now latch the anchor, $\boldsymbol{A} = \boldsymbol{L}(\theta_0)\boldsymbol{C}^{\mathrm{T}}$:

$$\boldsymbol{A} = \begin{pmatrix} -0.173648 & -0.984808 & 0 \\ 0.984808 & -0.173648 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{35}$$

which is $R_z(100°)$ exactly. That is Proposition 5.3 in numbers: the anchor comes out equal to the planetarium frame already in force, so nothing moves at the crossing. Measured as a rotation between the two frames, the difference is 5.3×10^{-15} degrees, which is round-off.

7.3 Holding the frame

Advance by $e = 90°$ of elapsed rotation, a little over five hours of Kerbin's day, and rebuild both frames:

quantity	value	by
$\boldsymbol{Z}$ turned	90.000000°	construction
$\boldsymbol{B}$ moved	1.6×10^{-14}°	Theorem 5.1
the pole moved	0°	Corollary 6.3

The sky has gone round a quarter turn and the body has not moved at all. The pole figure is exact rather than merely small: $\boldsymbol{Z}(e)^{\mathrm{T}}\boldsymbol{p} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{p}$ identically, so the two vectors are equal bit for bit and the angle between them is 0, not 10^{-14}.

7.4 Leaving the rotating frame

The vessel climbs back out. The flag clears, and three things follow from Equation 20 without any further machinery.

Nothing writes $\boldsymbol{Z}$ any more, since only the rotating branch does, so the sky freezes exactly where it stood. The anchor is dropped, so the next entry latches afresh rather than resuming this one. And $\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{L}(\theta)$ goes on being rebuilt every tick with $\boldsymbol{Z}$ now constant, so the body starts turning again.

Continuing the same example, coast a further 60° of Kerbin's rotation:

quantity	value
$\boldsymbol{B}$ at the exit tick, against $\boldsymbol{B}(e)$	2.3×10^{-14}°
$\boldsymbol{Z}$ drift over the coast	1.8×10^{-14}°
$\boldsymbol{B}$ turned	60.000000°
the pole moved	0°

Leaving costs nothing, which is the easy direction: freezing a quantity is always continuous, and the frames on either side of the exit agree to round-off. The body then turns by exactly the 60° its own spin phase advanced, and it turns **about its own pole**, which does not move. That last row is the whole point of the construction seen from the other side: in the rotating frame the sky went round the pole while the body stood still, and out of it the body goes round the pole while the sky stands still. The axis is the same one in both.

> **Why the anchor has to be dropped.** Suppose it were not, and the vessel came back below the threshold after that coast. The elapsed angle would be measured from the original latch, $\theta - \theta_0$, which has grown by the whole inertial arc, and $\boldsymbol{Z}$ would jump the moment the frame was re-entered:

re-entry with the anchor released	$2.1 \times 10^{-14\circ}$
re-entry resuming the stale anchor	60.000000°

The jump is the coast, exactly: 60° of sky in one tick, carrying every on-rails vessel with it. Note that Equation 20 does not prevent this on its own: it says what the anchor **is** once latched, and dropping it at the exit is a separate obligation on the implementation. Note too the asymmetry, which is why the symptom is one-directional: entering can jump, leaving cannot.

7.5 What the naive construction does instead

At the same instant, $\boldsymbol{B}_{\text{naive}} = T\,R_z(\theta_0 - \theta_{\text{inv}})$ puts the pole at

$$\boldsymbol{B}_{\text{naive}}\boldsymbol{e}_3 = \begin{pmatrix} -0.087156 \\ 0.340719 \\ 0.936117 \end{pmatrix}, \qquad \boldsymbol{B}\boldsymbol{e}_3 = \begin{pmatrix} 0.350677 \\ 0.026666 \\ 0.936117 \end{pmatrix}, \tag{36}$$

31.258274° apart. Proposition 6.2 predicts

$$\Delta = 2\arcsin(\sin 20.590672^\circ \cdot |\sin 50^\circ|) = 31.258274^\circ, \tag{37}$$

to every digit printed. Note the left-hand vector: it is $\boldsymbol{L}(\theta_0)\boldsymbol{e}_3$ unchanged, which is the failure exactly as Section 6 describes it. The naive pole never notices that the world frame has turned.

At $\theta_{\text{inv}} = 180^\circ$ the same formula gives 41.181343°, which is where the "about 41°" of Section 6 comes from.

7.6 The untilted case

Set $T = \boldsymbol{I}$ and $\mu = 0$, keeping everything else. Then $\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{L}(\theta)$ should be stock's $R_z(\theta_0 - \theta_{\text{inv}}) = R_z(37.4^\circ)$, and it is:

$$\boldsymbol{B}\boldsymbol{e}_1 = \begin{pmatrix} 0.794415 \\ 0.607376 \\ 0 \end{pmatrix}, \qquad R_z(37.4^\circ)\boldsymbol{e}_1 = \begin{pmatrix} 0.794415 \\ 0.607376 \\ 0 \end{pmatrix}, \tag{38}$$

with a largest component difference of 3.3×10^{-16}, one unit in the last place of a double. This is Proposition 5.6 measured rather than argued, and it is why an untilted install is unaffected by the patch.

8 Orbital elements

The same algebra answers a quite different question: what inclination should a player type in, and against what plane is it measured?

8.1 The orbit frame

An orbit's orientation is described by three angles: the longitude of the ascending node Ω, the inclination i, and the argument of periapsis ω. KSP builds their frame with the same generator as everything else:

$$\boldsymbol{O}(\Omega, i, \omega) = \boldsymbol{S}(\Omega, i, \omega) = R_z(\Omega)R_x(i)R_z(\omega). \tag{39}$$

The columns have direct meaning. By Equation 8 the third column is

$$\boldsymbol{O}\boldsymbol{e}_3 = \begin{pmatrix} \sin\Omega\sin i \\ -\cos\Omega\sin i \\ \cos i \end{pmatrix}, \tag{40}$$

the orbit normal — the direction of the angular momentum vector — and the first column points at periapsis. The reference plane is the plane $z = 0$, so the inclination i in Equation 40 is measured from **the celestial equator**, and Ω from **the celestial x-axis**.

8.2 Changing the reference plane

KSP measures every orbit against the celestial equator, the plane $z = 0$. For a system built from real data that is a workable choice, since ephemeris services will supply elements in that frame on request. They will supply them in others too: JPL Horizons returns the same orbit against the ecliptic, against the ICRF equator, or against a body's own equator, and the three disagree, so elements taken from one plane and read against another put the orbit somewhere it never was. For an invented system the celestial equator is simply a nuisance. What a designer means by "a moon in my gas giant's equatorial plane" is a statement about **the giant's** equator, and the celestial-frame numbers that express it are not round.

The fix is a single composition.

> **Proposition 8.1.** Let a parent body have tilt frame T. If (Ω, i, ω) are read against the parent's equator, the corresponding celestial-frame elements are those of
>
> $$\boldsymbol{O}_{\text{cel}} = T \cdot \boldsymbol{O}(\Omega, i, \omega), \tag{41}$$
>
> and conversely $\boldsymbol{O}_{\text{loc}} = T^{\mathrm{T}} \cdot \boldsymbol{O}_{\text{cel}}$.

Proof. The columns of $\boldsymbol{O}(\Omega, i, \omega)$ are written in the parent's equatorial basis. By Equation 17, T is precisely the matrix whose columns are that basis expressed in celestial coordinates, so T applied to any set of parent-frame coordinates returns celestial coordinates. Applying it to each column of $\boldsymbol{O}$ is matrix multiplication. The converse follows from $T^{-1} = T^{\mathrm{T}}$. □

Because both frames come from the same generator Equation 5, the product is again a frame of the same kind, and we can read elements back off it (Section 8.3).

> **Remark.** The composition uses T, the pole **alone**, and not $TR_z(\mu)$. The longitude of the ascending node is measured from an inertial reference direction; folding in the prime meridian would tie the orbit to wherever the parent's surface happens to put longitude zero, which has nothing to do with orbits.

The same idea rebases a **tilt** rather than an orbit: to give a moon a pole expressed relative to its parent's, carry the local pole through the parent's frame and re-read its right ascension and declination,

$$\boldsymbol{p}_{\text{cel}} = T_{\text{parent}} \boldsymbol{p}_{\text{loc}}, \quad \delta = \arcsin(p_3), \quad \alpha = \operatorname{atan2}(p_2, p_1). \tag{42}$$

8.3 Recovering elements from a frame

Inverting Equation 39 means reading Ω, i and ω back off the matrix entries. From Equation 8 and the general form in Proposition 2.1,

$$\boldsymbol{O}_3 = \begin{pmatrix} \sin\Omega \sin i \\ -\cos\Omega \sin i \\ \cos i \end{pmatrix}, \qquad \boldsymbol{O}_{13} = \sin i \sin\omega, \quad \boldsymbol{O}_{23} = \sin i \cos\omega, \tag{43}$$

where $\boldsymbol{O}_{13}$ means the third component of the first column. Hence, when $\sin i \neq 0$,

$$i = \operatorname{atan2}(\sin i, \cos i), \quad \Omega = \operatorname{atan2}(\boldsymbol{O}_{31}, -\boldsymbol{O}_{32}), \quad \omega = \operatorname{atan2}(\boldsymbol{O}_{13}, \boldsymbol{O}_{23}), \tag{44}$$

with $\sin i = \sqrt{\boldsymbol{O}_{31}^2 + \boldsymbol{O}_{32}^2}$ taken from the first two components of the third column. Section 8 explains why it must be computed that way and not any other.

8.4 The degenerate cases

> **Proposition 8.2.** If $i = 0$ then $\boldsymbol{O} = R_z(\Omega + \omega)$; if $i = 180°$ then $\boldsymbol{O} = R_z(\Omega - \omega)R_x(180°)$. In both cases Ω and ω are individually undetermined: only their sum, respectively their difference, is recoverable.

Proof. For $i = 0$, $R_x(0) = \boldsymbol{I}$ and Equation 4 gives $\boldsymbol{O} = R_z(\Omega)R_z(\omega) = R_z(\Omega + \omega)$. For $i = 180°$, note that $R_x(180°) = \mathrm{diag}(1, -1, -1)$ satisfies $R_x(180°)R_z(\omega) = R_z(-\omega)R_x(180°)$, since conjugating a z-rotation by a half-turn about x reverses it. Therefore $\boldsymbol{O} = R_z(\Omega)R_x(180°)R_z(\omega) = R_z(\Omega - \omega)R_x(180°)$. □

This is not a numerical artefact but a genuine geometric fact: an orbit lying in the reference plane has no ascending node, so there is nothing for Ω to measure. The implementation resolves it by convention, setting $\Omega = 0$ and putting the whole angle into ω. With $\Omega = 0$ the frame is $R_x(i)R_z(\omega)$, whose first column is $(\cos\omega, \cos i \sin\omega, \sin i \sin\omega)^{\mathrm{T}}$, so

$$\omega = \mathrm{atan2}(\boldsymbol{O}_{21} \cos i, \boldsymbol{O}_{11}). \tag{45}$$

The factor $\cos i$ is doing real work: since $\sin i = 0$ we have $\cos^2 i = 1$, so $\boldsymbol{O}_{21} \cos i = \cos^2 i \sin\omega = \sin\omega$, and multiplying by $\cos i = \pm 1$ absorbs the sign flip between the prograde and retrograde cases in one expression.

9 Numerical conditioning

Everything above is exact arithmetic. In double precision two of the formulas are traps, and each of them has caused a real bug in this implementation.

9.1 `acos` cannot resolve a small angle

Suppose $c = \cos i$ is known with absolute error η. Differentiating $i = \arccos(c)$ gives

$$\frac{\mathrm{d}i}{\mathrm{d}c} = -\frac{1}{\sqrt{1 - c^2}} = -\frac{1}{\sin i}, \tag{46}$$

so the error in i is about $\eta / \sin i$, which is unbounded as $i \to 0$ or $180°$. Near $i = 180°$ write $i = 180° - \beta$, with β in radians; then $\sin i \approx \beta$ and the error in the recovered angle is about η/β. That error is small next to β itself only while $\beta \gg \sqrt{\eta}$, and the two become equal at $\beta = \sqrt{\eta}$ exactly. With $\eta \approx 2 \times 10^{-16}$ the floor is

$$\sqrt{\eta} \approx 1.5 \times 10^{-8} \text{ radians}, \tag{47}$$

and below it `acos` does not resolve the angle at all: what it returns is no larger than its own error. At $\beta = 10^{-8}$ the error is already 2.2 times the angle being measured. Note where the loss occurs: $\arccos(-1)$ is exactly $180°$ in IEEE arithmetic, so a frame that arrives with $\boldsymbol{O}_{33}$ exactly -1 is fine. The damage is done upstream. A frame that is **mathematically** polar but reaches the decomposition as a product of other frames arrives with $\boldsymbol{O}_{33}$ a little above -1, and `acos` turns that into something like $179.999999°$.

The two-argument $\mathrm{atan2}(\sin i, \cos i)$ has no such problem: it uses both components, and near the poles the **sine** is the small, well-resolved quantity while the cosine is order one.

9.2 The cancellation that destroys the degeneracy test

Far worse is computing $\sin i = \sqrt{1 - c^2}$. Suppose the true frame is exactly retrograde-equatorial, so $\boldsymbol{O}_3 = (0, 0, -1)^{\mathrm{T}}$, but it arrived as a **product** of frames — as it does in Proposition 8.1 — and carries rounding. Suppose that rounding leaves $c = -1 + \eta$; the exact η depends on the operands, but a couple of units in the last place is typical, so take $\eta \sim 2 \times 10^{-16}$. Then

$$1 - c^2 = 1 - (1 - 2\eta + \eta^2) = 2\eta - \eta^2 \approx 4 \times 10^{-16}, \tag{48}$$

and the square root returns

$$\sqrt{1-c^2} \approx 2 \times 10^{-8}. \tag{49}$$

The true value of $\sin i$, computed from the first two components of the column, is of order 10^{-16}. **The subtraction has inflated the noise by eight orders of magnitude**: the classic signature of catastrophic cancellation, where subtracting two nearly equal quantities promotes rounding error into the leading digits.

Now trace the consequence. The implementation branches on whether the orbit is equatorial:

```
if (sinInc > 1e-12) { /* general case */ } else { /* equatorial convention */ }
```

With $\sin i$ reported as 2×10^{-8}, the near-degenerate frame takes the **general** branch, and Equation 44 recovers Ω from atan2 of two components that are pure rounding noise. The node comes back essentially at random. And by Proposition 8.2 that is not self-cancelling: for a retrograde equatorial orbit only $\Omega - \omega$ is determined, so splitting it wrongly between the two **moves the orbital plane**, by as much as 180°.

9.3 The fix

Take $\sin i$ from the column's own length in the equatorial plane, and the angle from atan2:

```
var cosInc = Clamp(frame.Z.z, -1.0, 1.0);
var sinInc = Math.Sqrt(frame.Z.x * frame.Z.x + frame.Z.y * frame.Z.y);
elements.Inclination = Math.Atan2(sinInc, cosInc) * Rad2Deg;
```

Both quantities are now computed from data that is small when the answer is small. No subtraction of nearly equal numbers occurs anywhere, the 10^{-12} gate becomes meaningful again, and the degenerate branch is entered exactly when it should be.

> **Moral.** $\sqrt{1-\cos^2 x}$ and $|\sin x|$ are equal in $\mathbb{R}$ and are **not** interchangeable in floating point. Prefer the form whose inputs vanish with the output.

10 Summary

The construction in one page.

object	definition
tilt frame	$T = \mathrm{PlanetaryFrame}(\alpha, \delta, 0)$, so $T\boldsymbol{e}_3 = \boldsymbol{p}$
local body frame	$\boldsymbol{L}(\theta) = TR_z(\theta + \mu)$ — where the planet points, inertially
body frame	$\boldsymbol{B} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{L}(\theta)$ — where it is drawn
planetarium frame	$\boldsymbol{Z}(e) = TR_z(e)T^{\mathrm{T}}\boldsymbol{A}$ — a spin about $\boldsymbol{p}$, by Cor. 4.2
anchor	$\boldsymbol{A} = \boldsymbol{L}(\theta_0)\boldsymbol{C}^{\mathrm{T}}$ — the unique choice freezing $\boldsymbol{B}$
element rebase	$\boldsymbol{O}_{\mathrm{cel}} = T\boldsymbol{O}_{\mathrm{loc}}$, inverse $\boldsymbol{O}_{\mathrm{loc}} = T^{\mathrm{T}}\boldsymbol{O}_{\mathrm{cel}}$

The three results that make it work:

1. **Theorem 5.1** — $\boldsymbol{B}$ is independent of elapsed rotation, because the conjugation by T puts the sky's $R_z(-e)$ and the body's $R_z(+e)$ on a common axis where they cancel.
2. **Proposition 3.1** — continuity at the threshold is a property of the invariant $\theta = \theta_{\mathrm{dir}} + \theta_{\mathrm{inv}}$, not of the generator, so any frame function inherits it.

3. **Proposition 5.6** — with $T = \boldsymbol{I}$ everything collapses to stock's own expressions exactly, so an untilted install is unaffected.

And the two traps:

1. **Proposition 6.1** — you cannot bolt a tilt onto stock's scalar arithmetic, because $R_z(-\theta_{\text{inv}})$ is only a change of coordinates when the two rotations share an axis.
2. **Section 9** — you cannot recover an inclination with `acos` and $\sqrt{1-c^2}$ near the degeneracies, because both destroy exactly the information the branch below them depends on.

Appendix A: Notation

symbol	meaning
$\boldsymbol{M}^{\mathrm{T}}$	transpose; equals $\boldsymbol{M}^{-1}$ for a frame
$R_z(\theta), R_x(\theta)$	elementary rotations, Equation 3
$\boldsymbol{S}(A, B, C)$	the ZXZ generator $R_z(A)R_x(B)R_z(C)$, Equation 5
$\boldsymbol{e}_3$	the reference z-axis, $(0, 0, 1)^{\mathrm{T}}$
t	universal time, seconds
P	`rotationPeriod`, the body's rotation period, seconds
θ	`rotationAngle`, the body's true spin phase, Equation 14
θ_{dir}	`directRotAngle`, per body, the spin phase in world coordinates
θ_{inv}	`InverseRotAngle`, global, how far the world frame has turned
e	elapsed rotation since entering the rotating frame
α, δ	right ascension and declination of the pole
$\varepsilon = 90° - \delta$	obliquity
μ	prime meridian offset
T	the tilt frame, $\mathrm{PlanetaryFrame}(\alpha, \delta, 0)$, Equation 17
$\boldsymbol{p} = T\boldsymbol{e}_3$	the body's north pole, in celestial coordinates
$\boldsymbol{C}$	the body frame at the instant the rotating frame was entered
Ω, i, ω	longitude of ascending node, inclination, argument of periapsis

All angles in this document are in degrees, matching what KSP stores and what its configuration files accept. In the source the split is: `PlanetaryFrame` and `OrbitalFrame` take degrees and convert on entry, while the `SetFrame` they both call takes radians.

Appendix B: Proof of the conjugation lemma

Restating Lemma 4.1: for a rotation $\boldsymbol{R}$ and a rotation $\boldsymbol{R}_{\boldsymbol{u}}(\theta)$ by θ about the unit axis $\boldsymbol{u}$,

$$\boldsymbol{R}\,\boldsymbol{R}_{\boldsymbol{u}}(\theta)\,\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{R}_{\boldsymbol{Ru}}(\theta). \tag{50}$$

Two standard facts about rotations are used in the proof. First, a rotation acts on the plane perpendicular to its own axis as an ordinary planar rotation: for unit $\boldsymbol{u}$ and any unit $\boldsymbol{w}$ perpendicular to it,

$$\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{w} = \cos\theta\,\boldsymbol{w} + \sin\theta\,(\boldsymbol{u}\times\boldsymbol{w}), \tag{51}$$

which is the familiar two-dimensional formula written in the orthonormal pair $(\boldsymbol{w}, \boldsymbol{u}\times\boldsymbol{w})$ spanning that plane. Second, a rotation preserves cross products:

$$(\boldsymbol{M}\boldsymbol{a})\times(\boldsymbol{M}\boldsymbol{b}) = \boldsymbol{M}(\boldsymbol{a}\times\boldsymbol{b}) \qquad \text{for all } \boldsymbol{M}\in\mathrm{SO}(3). \tag{52}$$

Both sides are linear in $\boldsymbol{a}$ and in $\boldsymbol{b}$, so it suffices to check them on basis vectors, where the statement reduces to the columns of $\boldsymbol{M}$ forming a right-handed triple. It is here that $\det\boldsymbol{M} = +1$ matters: a reflection would flip the sign.

Proof. Write $\boldsymbol{N} = \boldsymbol{R}\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{R}^{\mathrm{T}}$. First, $\boldsymbol{N}$ is itself a rotation, since

$$\boldsymbol{N}\boldsymbol{N}^{\mathrm{T}} = \boldsymbol{R}\boldsymbol{R}_{\boldsymbol{u}}(\theta)\underbrace{\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R}}_{\boldsymbol{I}}\boldsymbol{R}_{\boldsymbol{u}}(\theta)^{\mathrm{T}}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{R}\underbrace{\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{R}_{\boldsymbol{u}}(\theta)^{\mathrm{T}}}_{\boldsymbol{I}}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}, \tag{53}$$

and $\det\boldsymbol{N} = \det\boldsymbol{R}\cdot\det\boldsymbol{R}_{\boldsymbol{u}}(\theta)\cdot\det\boldsymbol{R}^{\mathrm{T}} = 1$.

Now fix any unit $\boldsymbol{w}$ perpendicular to $\boldsymbol{u}$. Then $(\boldsymbol{u}, \boldsymbol{w}, \boldsymbol{u}\times\boldsymbol{w})$ is a right-handed orthonormal basis, and since $\boldsymbol{R}$ preserves lengths, angles and (by Equation 52) cross products, so is $(\boldsymbol{Ru}, \boldsymbol{Rw}, \boldsymbol{Ru}\times\boldsymbol{Rw})$. Two linear maps are equal when they agree on a basis, so it is enough to compare $\boldsymbol{N}$ with $\boldsymbol{R}_{\boldsymbol{Ru}}(\theta)$ on this one.

On the axis. Using $\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{I}$ and the fact that $\boldsymbol{u}$ is fixed by $\boldsymbol{R}_{\boldsymbol{u}}(\theta)$,

$$\boldsymbol{N}(\boldsymbol{Ru}) = \boldsymbol{R}\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{u} = \boldsymbol{Ru}, \tag{54}$$

and $\boldsymbol{R}_{\boldsymbol{Ru}}(\theta)$ fixes $\boldsymbol{Ru}$ by definition. Both agree.

On the perpendicular. Applying Equation 51 and then Equation 52,

$$\boldsymbol{N}(\boldsymbol{Rw}) = \boldsymbol{R}\boldsymbol{R}_{\boldsymbol{u}}(\theta)\boldsymbol{w} = \boldsymbol{R}(\cos\theta\,\boldsymbol{w} + \sin\theta\,(\boldsymbol{u}\times\boldsymbol{w})) = \cos\theta\,(\boldsymbol{Rw}) + \sin\theta\,(\boldsymbol{Ru}\times\boldsymbol{Rw}). \tag{55}$$

Since $\boldsymbol{Rw}$ is a unit vector perpendicular to $\boldsymbol{Ru}$, this is exactly what Equation 51 gives for $\boldsymbol{R}_{\boldsymbol{Ru}}(\theta)$ applied to $\boldsymbol{Rw}$. Both agree, **and with the same sign on the $\sin\theta$ term**, which is what settles the sense of the rotation rather than merely its magnitude.

On the third basis vector. Both maps are rotations, so both preserve cross products by Equation 52. Their images of $\boldsymbol{Ru}\times\boldsymbol{Rw}$ are therefore determined by their images of $\boldsymbol{Ru}$ and $\boldsymbol{Rw}$, which already agree.

The two maps agree on a basis, hence are equal. □

Appendix C: The swizzle

KSP's celestial coordinates are right-handed with z up; Unity's are left-handed with y up. Conversion swaps the last two components,

$$\boldsymbol{S}_{\mathrm{w}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \boldsymbol{S}_{\mathrm{w}} = \boldsymbol{S}_{\mathrm{w}}^{\mathrm{T}} = \boldsymbol{S}_{\mathrm{w}}^{-1}, \tag{56}$$

which is what `Vector3d.xzy` computes. Note $\det\boldsymbol{S}_{\mathrm{w}} = -1$: the swap changes handedness, which is exactly what is wanted, since the two conventions differ in handedness too.

A frame is carried across by conjugation, $\boldsymbol{M} \mapsto \boldsymbol{S}_{\mathrm{w}}\boldsymbol{M}\boldsymbol{S}_{\mathrm{w}}$, which is the map `QuaternionD.swizzle` implements. Two properties matter:

1. It is a **homomorphism**: $\boldsymbol{S}_{\mathrm{w}}(\boldsymbol{M}\boldsymbol{N})\boldsymbol{S}_{\mathrm{w}} = (\boldsymbol{S}_{\mathrm{w}}\boldsymbol{M}\boldsymbol{S}_{\mathrm{w}})(\boldsymbol{S}_{\mathrm{w}}\boldsymbol{N}\boldsymbol{S}_{\mathrm{w}})$, since $\boldsymbol{S}_{\mathrm{w}}\boldsymbol{S}_{\mathrm{w}} = \boldsymbol{I}$. So **composition order is preserved**, and every identity proved in this document survives the conversion unchanged.
2. It preserves determinants, so rotations map to rotations despite $\det \boldsymbol{S}_{\mathrm{w}} = -1$.

Consequently all of the algebra above may be done entirely in celestial coordinates, converting only at the boundary where a value is handed to Unity.

Appendix D: Where this lives in the code

result	**implementation**
Equation 17, Prop. 2.2	`TiltEmFrames.FromPole`
$\boldsymbol{L}$, Prop. 2.3	`TiltEmFrames.LocalBodyFrame`
$\boldsymbol{B}$, Prop. 3.2 generalised	`TiltEmFrames.BodyFrame`
$\boldsymbol{Z}$, Cor. 4.2	`TiltEmFrames.Zup`
$\boldsymbol{A}$, Cor. 5.2	`TiltEmFrames.AnchorFor`
Prop. 8.1	`TiltEmFrames.ToCelestialElements`, `ToLocalElements`
Pole rebase	`TiltEmFrames.ToCelestialTilt`
Section 8.3, Section 9	`TiltEmFrames.DecomposeOrbitalFrame`